KRPCLAB · BOOSTERLAND

How the booster lander works

Bringing a spent first stage back to the launch pad, Super Heavy style, from a kRPC script.

The booster separates at about 20-35 km, still moving downrange at several hundred metres per second. The autopilot flips it, burns back toward the pad, falls through the atmosphere steering with its body and braking with its airbrakes or fins, pushes sideways on its RCS, and lights the engines once more for a suicide burn that stands it upright on the pad.

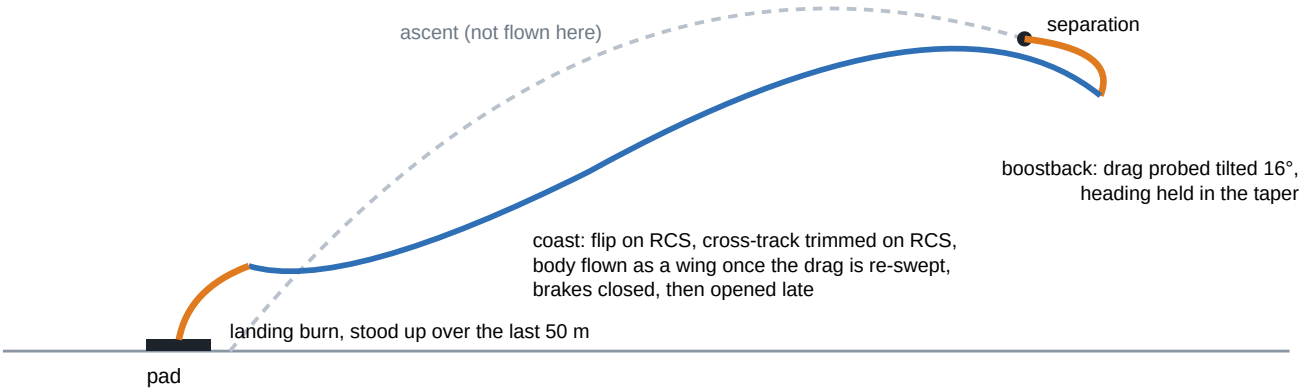


Figure 1. The flight profile, not to scale. Orange is engine-on, blue unpowered. Every part of it is re-decided each tick from a fresh prediction of where the booster will touch down.

The one idea: re-predict the landing every tick

There is no stored plan. About twenty times a second the autopilot reads the booster's position and velocity and integrates the rest of the flight to the ground in local Python: gravity, drag, body lift, the planet's rotation and the landing burn itself. Every decision that tick (where to point, how much throttle, when boostback ends, when the landing burn lights, how far the brakes open and at what height they open fully, which way the thrusters push) is taken from that fresh prediction.

`autoland.py` is the phase machine and the only code that talks to the game; `trajectory.py` the propagator; `guidance.py` the control laws as pure functions; `engines.py` the throttle allocation; `environment.py` the cached world data (density, speed of sound, drag curves, lift slope, holdable angle, lift peak), so the integration makes no remote calls.

- **Through the burn.** The prediction coasts to where the suicide burn will start and keeps integrating with the engines on, through the same throttle law the vehicle flies (`trajectory.landing_command` is shared). RK4 throughout.
- **Drag** is a curve of Cd·A against Mach, probed with kRPC's `simulate_aerodynamic_force_at`, which evaluates the vessel at a chosen attitude with its parts as they are now, deploy angles included. While the booster is not yet falling nose-first (more than 8° off retrograde) every probe is taken with the vessel turned onto retrograde tilted 16° in the trajectory plane. Once it is within 8° of retrograde the whole curve is replaced by a sweep at the real attitude (the **descent re-sweep**), re-swept every 10 s and 2 s after the brakes move. A second curve, measured once with the brakes fully open, lets the propagator fly a brake schedule (below).
- **Lift** goes as aoa·|aoa| times a slope probed the same way, credited only up to the holdable angle, and discounted by the measured aoa tracking (held ÷ commanded).
- **The miss** is resolved into *long* (along the approach, positive past the pad) and *cross*, about the pad's own vertical.
- **Heights** are measured from the feet: the COM-to-feet offset is read from the bounding box at start and again on every phase change; terrain under the legs is clamped against the pad's radius.

- **The target** is the launch site the window mod reports, when it is on this body: the position recorded at this vessel's launch (keyed by its parts' launch ID, kept through FMRS and quicksaves), else the spawn point of the site it was launched from. Without one it is `PAD_LAT` / `PAD_LON`; the offset between the two is logged.

The flight, phase by phase

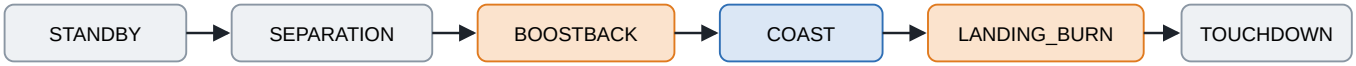


Figure 2. The phase machine. Orange phases burn the engines. Any phase ends the flight if the vessel is destroyed.

STANDBY waits for START	On START: RCS permitted; airbrakes deployed at 60°, the middle of the deploy law's range; on a booster with rings of four or more evenly spaced fins, alternate fins are deployed opposite ways as brakes (up to their own maximum deploy angle) and fly the same brake law; every liquid engine activated and grouped; kRPC's roll threshold set to 180°; the autopilot engaged on the current attitude. The leg-contact limit is read once (below).
SEPARATION a few seconds	`proximity.py` wraps each nearby craft in a sphere and marches the booster's motion forward against each, coasting and at full thrust along the boostback aim. The flip starts on the first tick nothing is in the way; a 5 s backstop flips anyway. Control surfaces are held neutral from here through boostback (their ignore flags set, brake-fin deploy kept), because at 50-150° off the flow kRPC's PID would swing them.
BOOSTBACK flip and burn	Aim opposite the predicted horizontal miss and go to full throttle before the turn is finished, so the gimbal does the turning; the flip is two pitches through the vertical (Figure 3). While full thrust along the nose would close on a neighbour, the throttle is held at zero. Throttle tapers with the remaining Δv. On the first tick the taper begins with the nose within 25° of the aim, that aim is held, and only the miss along it is burnt off (throttle = that miss ÷ time to land ÷ 60 m/s). The burn ends when the miss along the held heading has stayed within 190 m for four ticks; the cross-track is left to the coast.
COAST engines off	Surfaces handed back. Flip to retrograde (through the vertical again) with the RCS valve on the error relay until the nose is within 10° of its command (settled); then the cross-track is trimmed on the RCS (below). On the first tick a booster whose leg types are not all in the cache cycles its legs once to time them. Once the descent re-sweep has replaced the boostback drag curve the body is flown as a wing; until then it holds retrograde. The brakes first fly the late-brake schedule, then the ordinary deploy law. At the predicted ignition point → LANDING_BURN.
LANDING_BURN suicide burn	Gimbals stop answering roll; the RCS valve is held open. Throttle follows a constant-deceleration profile; steering is ZEM/ZEV on the propagator's predicted touchdown, rate-limited, stood upright over the last 50 m. Legs go down by their measured time.
TOUCHDOWN hand back	Engines cut on reported contact (or a hover a few metres up). The contact's vertical and horizontal speed are logged against the touchdown limits. Situation, distance and speed are recorded 2 s later; the booster is watched 8 s more for tilt. Every exit runs one shutdown: throttle and translation zero, autopilot off, SAS on, engine limits back to 100%, gimbal roll re-enabled, panel removed.

BOOSTBACK: the flip's throttle

Until the nose first comes within 25° of the aim, each tick the throttle is the smaller of the boostback's own throttle and the *u* that minimises the cost of the rest of the turn (`guidance.flip_throttle`):

$$\text{cost}(u) = u\cdot a\cdot t(u)\cdot(1-\sin\theta/\theta) + D\cdot t(u), \quad t(u) = (2\sqrt{(\alpha\theta+\omega^2/2)}-\omega)/\alpha, \quad \alpha = u\cdot\alpha_g + \alpha_f$$

- θ the angle left to turn, ω the rate it is closing; a the full-throttle acceleration; t(u) the bang-bang time to turn θ and stop; 1 − sin θ/θ the share of thrust that does not point along the aim over the turn.
- $D = v_{away}/T + x/T^2$: the return Δv each second of turning adds (x ground distance to the pad, T predicted time to land).
- α_g gimbal angular acceleration per unit throttle from the engines' geometry (Σ thrust × sin(range) × lever, × 0.9, over the weaker tilt axis' inertia); α_f RCS and wheels, read every tick (the largest RCS torque seen is kept).
- u is searched on a 0-1 grid in steps of 0.025; zero is allowed. `ascent/budget.boostback` integrates the same law, so the ascent's separation point plans for the flip this flies.

Attitude: flips through the vertical, roll held all flight

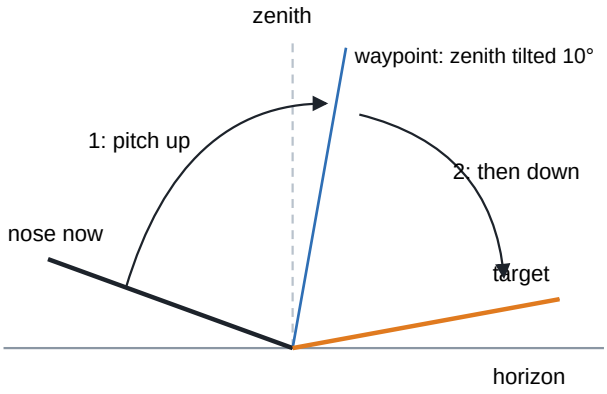


Figure 3. A turn over 90° is two pitches through the vertical; memoryless, it re-reads the real nose every tick.

Roll reference. `guidance.roll_reference` puts the pitch axis normal to the trajectory plane, the plane every steering command lies in. The first roof is taken on the side nearest the roof the vehicle has, so it never asks for more than 90° of roll; after that each tick's roof is the one carried by the nose's own rotation (`transport_roof`), so a flip over the top is pitch alone. With roll held, the cross-track direction is the vessel's right axis.

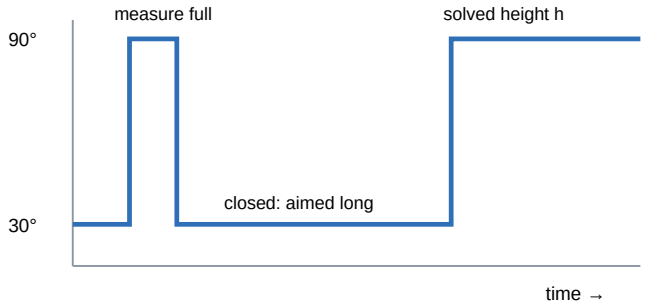
Who rolls it. RCS and wheels always; the engine gimbals from START until the landing burn, then pitch and yaw only. kRPC credits a cluster's gimbals with almost no roll torque, so its roll gains would spin the booster up at ignition.

Cross-track: a sideways push on the RCS

The coast pushes sideways on the RCS (`Autoland.run_rcs_trim`): armed once the flip has settled; a push starts when |cross| > 10 m and stops inside 3 m; the input is cross ÷ 30 m, capped at 1, against the cross-track's ground direction, turned into the vessel frame and sent as right, forward, up. While pushing, the RCS valve is held open (kRPC's gains follow the torque it believes available). Until one push has closed its miss the sign is checked: a cross-track grown 15 m after 4 s flips the sign once; a second failure switches the trim off. 100 m of cross-track at coast entry is ~0.7 m/s of sideways speed: a second or two of thrusters.

Two controls in the coast: lift and drag

Thrust after boostback can only shorten the flight. The air pushes both ways: the angle of attack makes a side force, and the brakes' deploy angle sets the drag. Drag carries the range; lift trims what is left.



deploy law (after the switch, or without it)

long = along-track miss of the unsteered prediction

S = Δlong / Δln(CdA) (two propagations, drag × 1.05)

k = Δln(CdA) / Δangle (curve before / after a step)

Δangle = −0.7 · long / (S·k), ≤ 20° a step, every 10 s

30-90°, deadband 30 m, re-sweep drag 2 s after each step

more open: more drag, lands shorter

Figure 4. The brakes. Left: once the booster falls nose-first and its drag has been re-swept, the brakes are opened fully once to measure the full-brake curve, closed, and held closed so the booster flies long; the propagator flies "closed until h, fully open below", h being the height that lands on the pad. Right: the Newton law that takes over once the full brakes are on.

Late brakes (`run_late_deploy`). Slower at ignition is a cheaper burn, so the coast arrives fast and spends its drag at the end. After the descent re-sweep, in descent attitude: brakes to 90°, every Mach bin probed and stored as the full curve; brakes to 30° and a re-sweep of the closed curve (if full has no more drag than closed, the ordinary law flies). Every 5 s a bisection on the unsteered prediction's along-track miss finds the switch height h to 10 m; every prediction flies the schedule; at h the brakes go to 90° and the propagator uses the full curve.

Deploy law (`run_deploy`). A Newton step every 10 s on a gain it measures itself, answering the unsteered prediction's along-track miss (the lift re-nulls the steered one every tick).

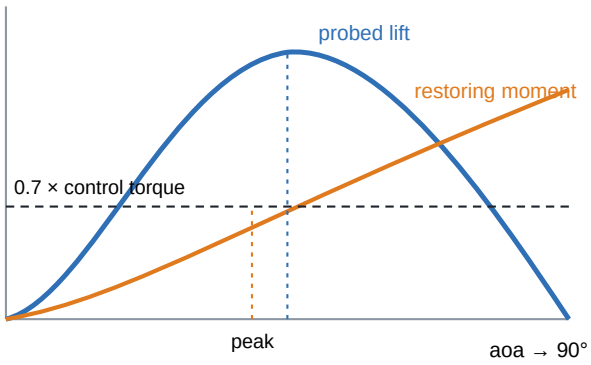


Figure 5. The wing's ceiling: the smaller of where the restoring moment exhausts the control torque and the angle of peak lift, per Mach bin.

The holdable angle. Every 2 s `refresh_holdable` turns the vessel to eight angles off retrograde, evenly spaced to 90°, and asks the game for the aerodynamic torque and force at each. The restoring moment per pascal stands against the control torque (wheels, the highest RCS torque seen, surfaces × q) × 0.7: where they meet is the most the booster can hold. The force gives the lift at the same angles; the angle of peak lift (the largest, refined by the parabola through its neighbours) caps it, because past the peak more angle buys less lift and the solve's model would run backwards. Both are kept per Mach bin. Until something is measured the wing does not steer.

Lift (once the descent re-sweep is in). Each tick: propagate small test angles in-plane and out-of-plane, invert the 2×2 to null long and cross, propagate the answer and keep it only if it lands nearer than doing nothing (else halve once, else hold retrograde). The command is capped at the holdable angle.

The landing burn: profile, steering, standing up

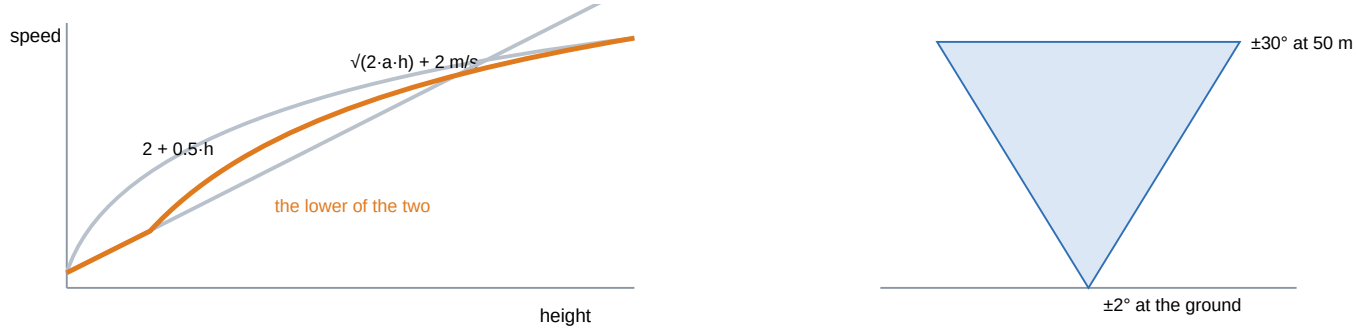


Figure 6. Left: the throttle tracks the lower of a constant-deceleration stop at 95% of available thrust and a constant-rate final descent. Right: below 50 m of leg height the whole steering command is held inside a cone round vertical that narrows to 2° at the ground.

Trigger and throttle. The burn lights when the height falls to the full-throttle stopping distance × 1.18 plus 1.5 s of fall. Feed-forward plus a trim of 6 m/s² per m/s of error, over the booster's own maximum acceleration. The tracked speed is $\sqrt{(v_{down}^2 + (w \cdot v_{side})^2)}$, w = height ÷ 250 m: above 250 m all of |v| is braked, near the ground only the descent rate. Below 250 m an ascending booster gets zero throttle: there the attitude law holds the thrust near vertical, so thrust while climbing only adds climb (above it the nose is retrograde and still brakes). Below 12 m at under 8 m/s the throttle is capped at hover + 0.12.

Steering. ZEM/ZEV, gains 6 and 4, on the predicted touchdown; tilt capped at 15° and at acos(throttle ÷ cap), the throttle opened by 1/cos(tilt). Below 250 m only the sideways velocity is nulled, from retrograde held inside 15° of vertical; below 50 m the command is clamped into the shrinking cone. The command may move no faster than a quarter second of the vehicle's own angular acceleration per second (kRPC's credited torque over inertia, measured while lit, never under 5°/s).

Landing legs and touchdown limits

Timed once, dropped by time to go. kRPC exposes neither a group's membership nor an animation's length, so on the first COAST tick a booster whose leg types are not all in `boosterland/data/gearTimes.json` lowers its gear, times each part to deployed, and raises it again. The gear then goes down when the time to go (height ÷ descent rate, or the prediction's time to land) is under 1.25 × the deploy time + 1 s; without a time, at the burn's first tick, with a 350 m backstop.

How hard it may land. At start, `common.touchdown.touchdown_limit` takes the weakest impact tolerance among the legs, or on a legless booster among the parts within 1 m of its lowest point. With the legs out (the timing cycle, or after gear down) one call to the window mod (`KrpcLabLegs.Legs`) returns each leg's live suspension: spring and damper as the game set them for the load, travel, foot position, friction, and `ModuleWheelDamage`'s impact tolerance. From those:

- **vertical:** a leg explodes when its down force jumps by its impact tolerance (× g_0/g) in one physics step; on contact the damper's force jumps from 0 to damper × v, so v_{max} = tolerance ÷ (damper + spring × 0.02 s), and no more than the stroke bottoming out at travel × √(springRatio × g);
- **lateral, foot held by friction:** the craft pivots about its leading feet with angular momentum conserved through the contact, and topples if that rotation lifts the COM over them; the lever is the COM's distance to the nearest edge of the feet's polygon, reduced by the tilt the springs give as the weight moves onto the pivot, atan((n/springRatio)/2b); and whether a sliding foot can tip it (friction × COM height > that lever).

Both are logged, and the contact is logged against them.

Engine allocation: one group at a time

At START each liquid engine's position and thrust are read and `engines.group_engines` groups them about the thrust-weighted centroid: within 0.3 m of the axis is the centre group; in a ring an engine is paired with the one

that most nearly cancels its offset; whatever cannot be paired lights together, so every group's thrust passes through the axis. A group stays dark until the one before it is at 100%; only the group in progress is part-throttled. Guidance reads `max_thrust`, so the planner sees the whole cluster.

Smaller rules

- **RCS valve:** held open in the landing burn and while the trim pushes; in COAST the error relay until settled, then open only when the controller is saturated (input ≥ 0.95 for 0.5 s) and not closing; elsewhere a relay on pointing error.
- **A destroyed vessel ends the flight:** its streams freeze, so a position bit-identical for 1 game-second while moving is a vessel lost; a mass drop over 10% in a tick is logged as parts lost.
- **Telemetry:** one line per tick with the miss (long/cross), Cd·A, Mach, commanded and held angle, holdable angle, lift slope, tracking, pointing error, inputs, engine groups; events for each phase change, leg clearance change, brake step, trim push, gear timing, launch site, lift peak and limits. Every log ends with the loop rate per phase; the config line carries the settings that differ from the defaults and a fingerprint of the defaults.

How it is measured

Each flight writes `logs/LOG<n>`, telemetry paced in game time. Tunables live only in `config.py`; `--set FIELD=VALUE` overrides one. Reference saves in `saves/` give the separation states (`saves/README.md`); `tools/savegen.py` writes more; `tests/fakeksp` runs the closed loop offline. `tools/sweep.py` flies arms × saves over the farm; for a booster that fell over, the reported distance is where the wreck lies.

State of the code on 2026-11-12 (defaults 680d8568). Tunables: `boosterland/config.py`.